

Title: Using CNN to detect Tomato leaf diseases classifications.

Author(s):

Name	Student ID
1. Artafat Islam	2022-3-60-303
2. Abir Mohd.Sajedul Hossain	2022-2-50-004
3. Bushra Jahan Monira	2022-3-60-241
4.Khadiza Ahmed Tayeba	2022-2-50-009

Acknowledgment: This work was carried out as part of the CSE366 Artificial Intelligence course under the supervision of Dr. Raihan Ul Islam.

Abstract : The plantVillage dataset have huge numbers of images. Initially it have potato, paper bell and tomato with different classes. We take only tomato , it has 10 classes. Each class have new diseases and one class is healthy. So, we have 9 classes sets with diseases and one with healthy. At first, Exploratory Data Analysis and related works summary. Visualized the sample images to understand class variation, image properties such as resolution, aspect ratio, and color distributions. Related work, we reads some papers on same topic and related to our dataset. We add references of these papers. We apply pre-trained CNN models (VGG16, ResNet50, EfficientNetB0) on our dataset by fine tuining and feature extraction. VGG16 model performed really well with 92.68% accuracy. We split the dataset in 3 sections. Training with 70% of the data from dataset, Validation with 10% data and test on 20% data from the dataset. Then we made our own custom CNN for better learning. It performed really well with 96.29% accuracy. Then we enhanced with GCNet. Lastly we use Grad-CAM to visualize the image and detect that where our model give more attention to classify an image and where not. We also apply our Custom CNN on other dataset of tomato diseases and got accuracy around 85%.

1.Introduction

We take PlatVillage dataset in Kaggle and our whole work is implemented in kaggle. We don't take the whole dataset , we take only tomatoes classes. We have extract the 10 classes of tomato and then we implement our works one by one. It's a agriculture base works. We work with 16,011 images in this project.

The main contributions of this work include:

- 1.Doing EDA and Related works to explore the dataset.
2. Use pre-trained Models like VGG16, ResNet50 and EfficientNetB0

3. Made our Own custom CNN for better learning and better classifications.
4. Enhanced the custom CNN with GCNet
5. Visualize predictions, highlighting image regions for correct and incorrect samples with Grad-CAM and apply our custom CNN on another tomato dataset. Then compare the values with our main dataset

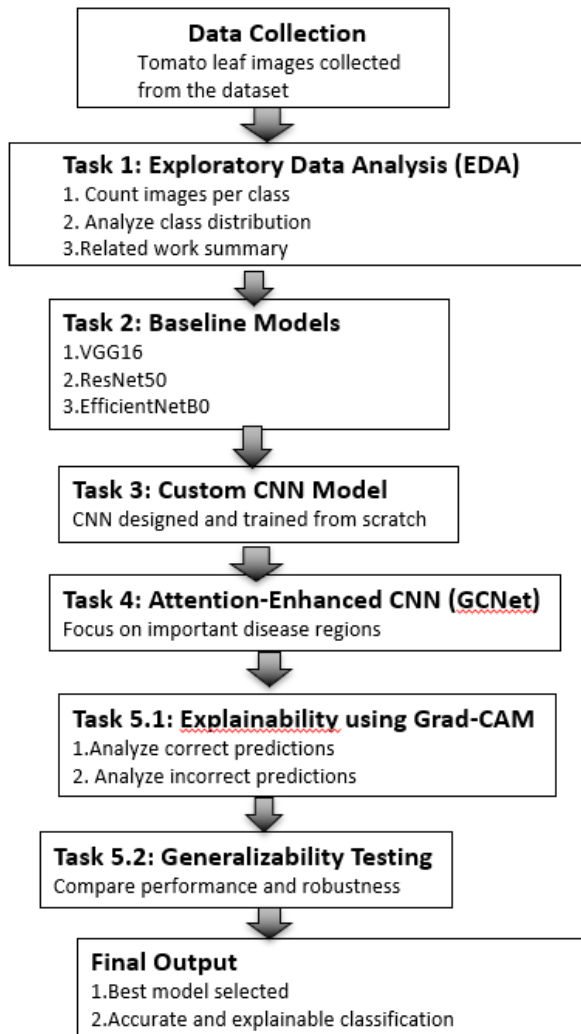
2.Methodology

2.1 Dataset Description

The PlantVillage dataset was used but focusing on ten tomato classes (nine diseased and one healthy). The dataset comprised approximately 16,011 images split into training (11,200), validation (1,604), and testing (3,207) subsets. Images were resized to 256×256 pixels and normalized for consistent input. Data augmentation (rotation, flipping, and zoom) was applied to improve generalization.

2.2 Methodology Flow

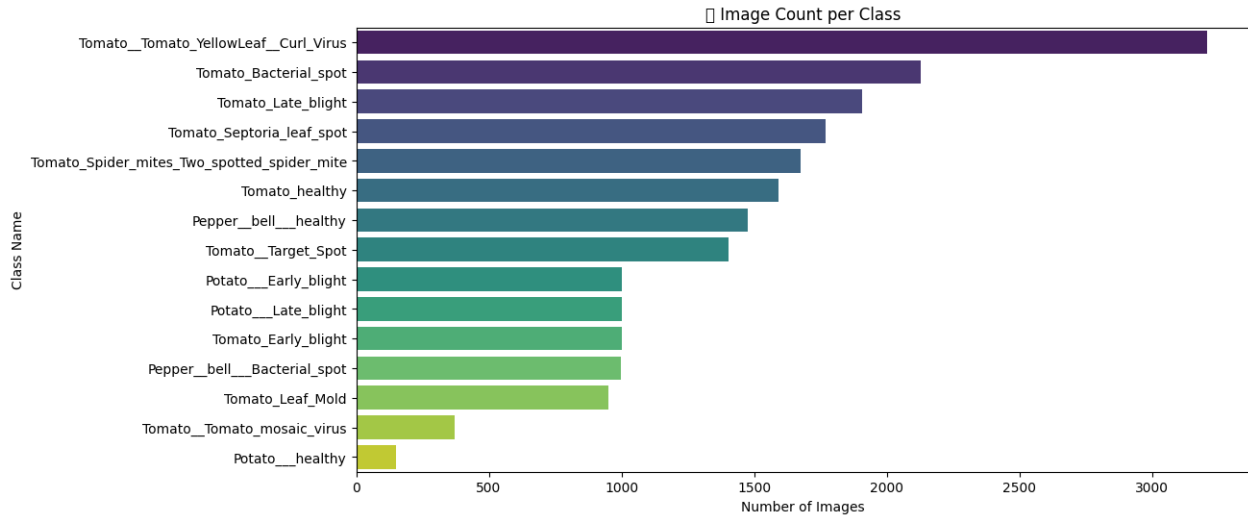
Tomato Leaf Disease Classification Workflow



(Refer to Figure : Methodology Flowchart)

2.3. Evaluation

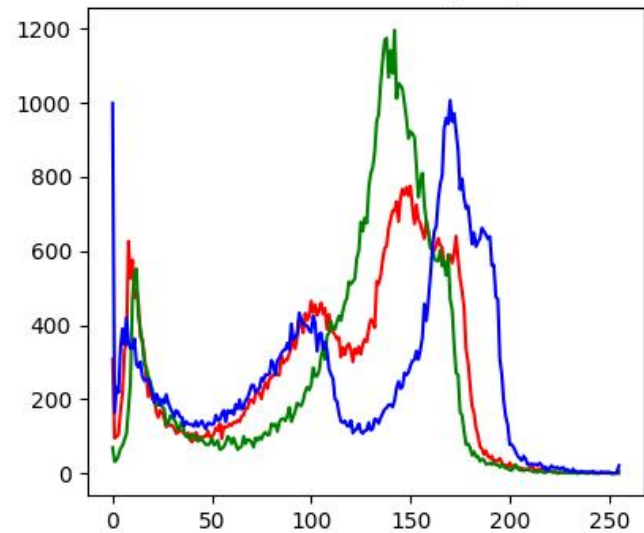
Task 1 : Exploratory Data Analysis (EDA) and Related Work



Sample Image: Tomato_Septoria_leaf_spot



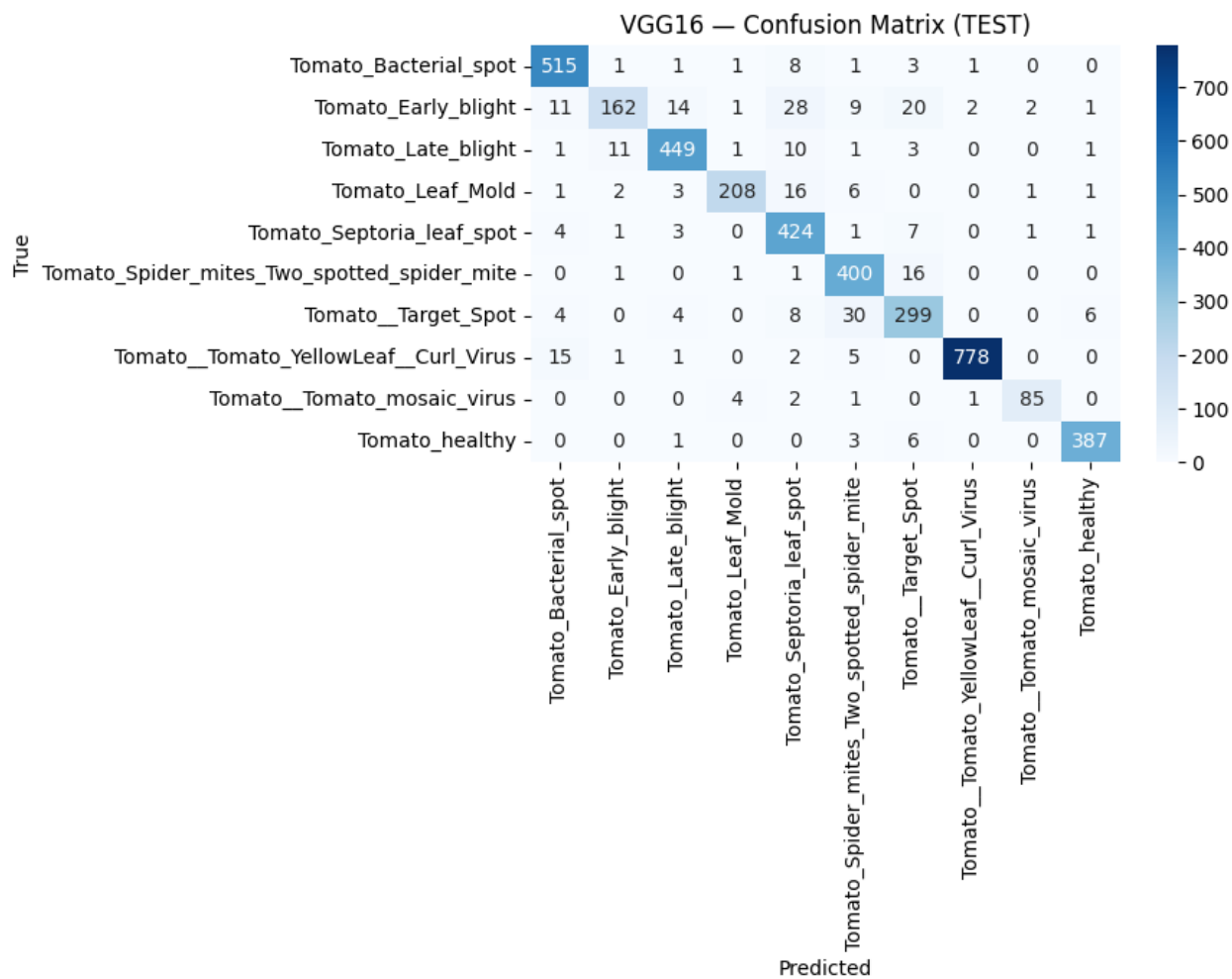
Color Distribution (RGB)



Task 2: Supervised Pre-trained Baselines

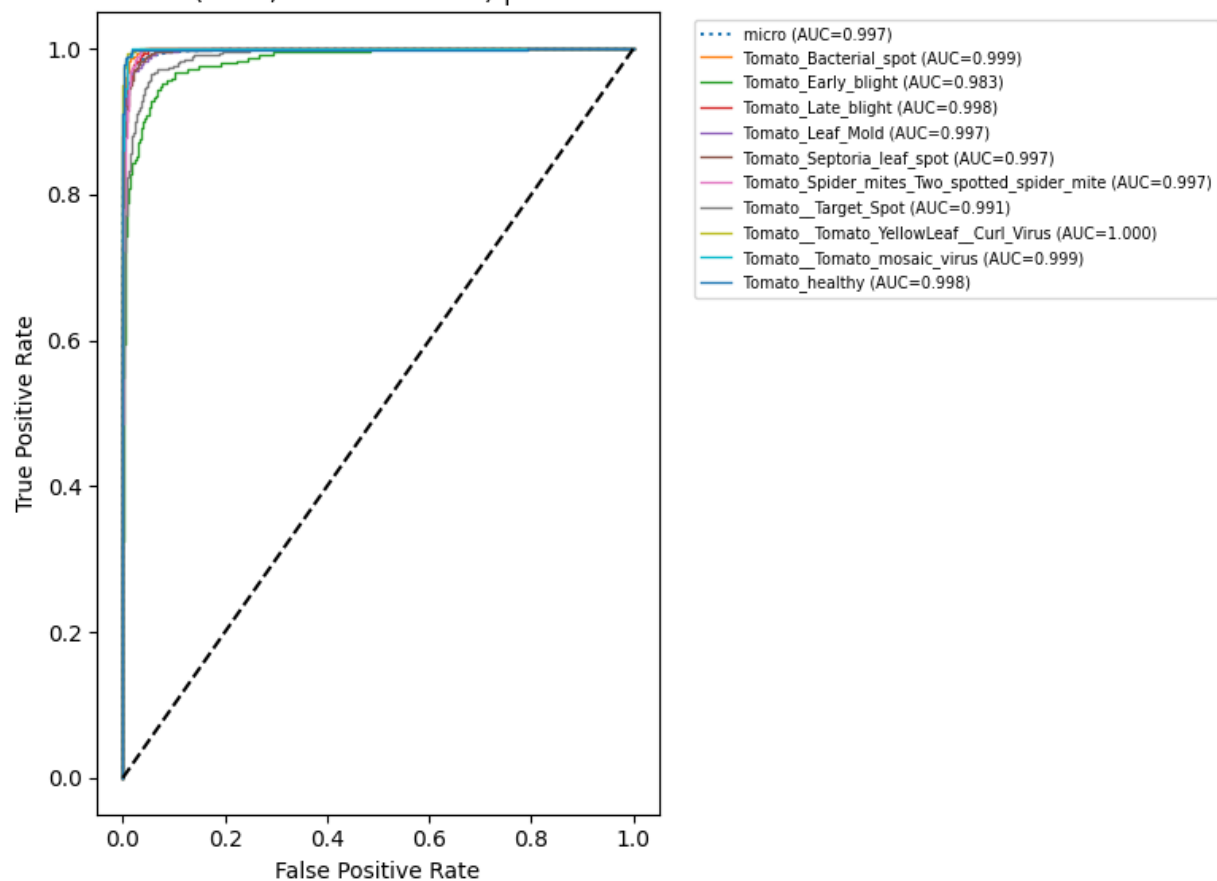
(a). Using transfer learning by freezing initial layers and fine-tuning the final layers 3 pre-trained CNN models. We Maintain consistent training parameters across all models for fair comparison. All pre-trained models have same ratio of train,val and test, same epochs, same learning rate, same patience etc.

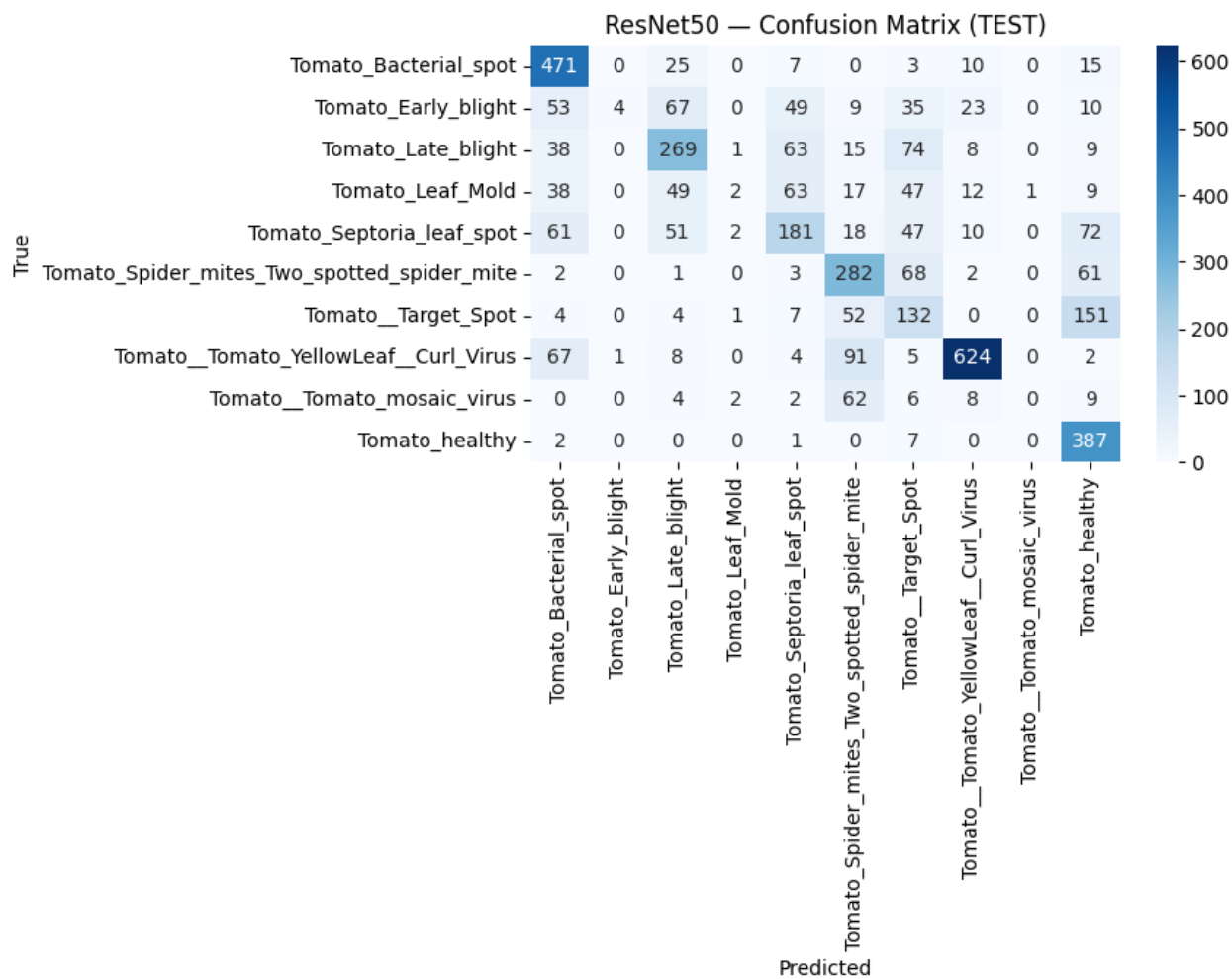
(b) **Metrics:** Report Accuracy, Class Accuracy, Precision, Recall, F1-score, Confusion Matrix, AUC, ROC curve, and training/testing time. Used a paired t-test for statistical comparison.



VGG16—TEST Acc:92.67%,Source:imagenet

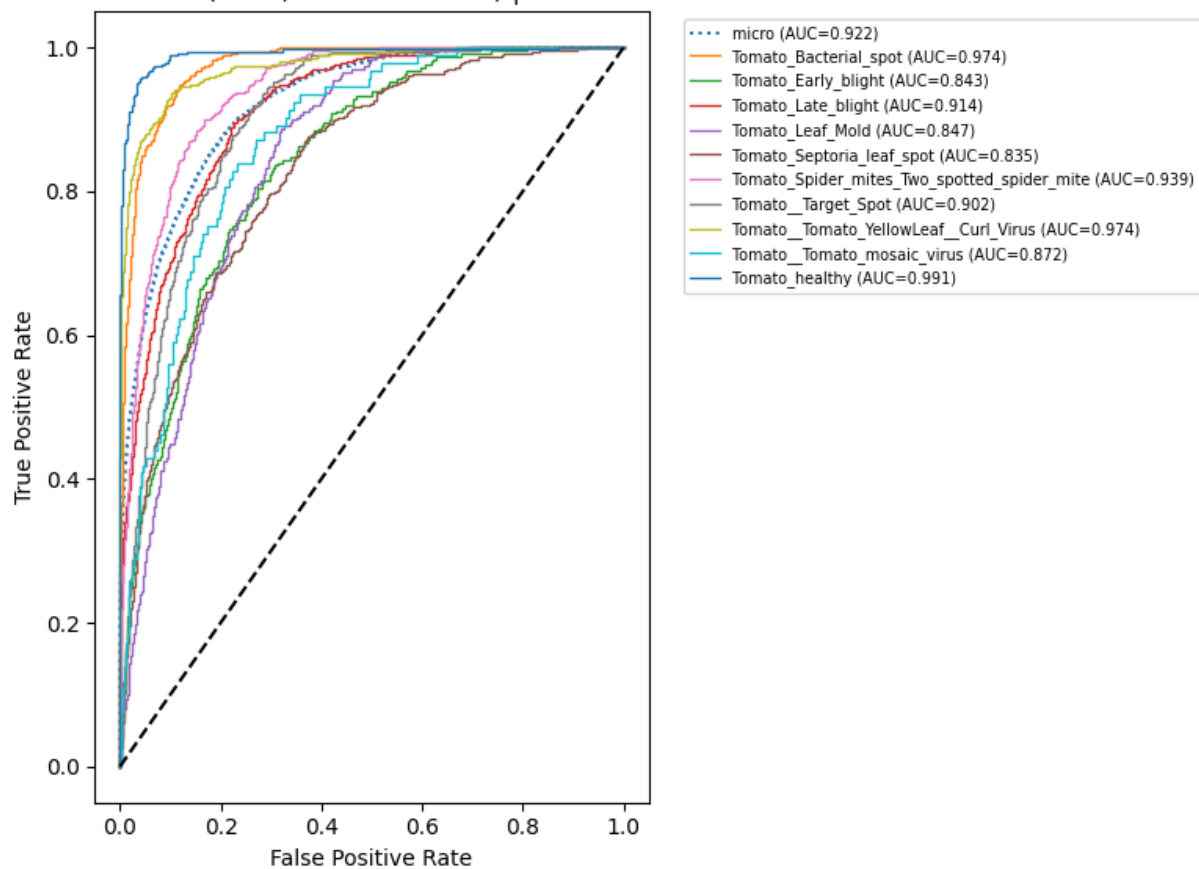
VGG16 — ROC (TEST, first 10 classes) | macro AUC=0.996

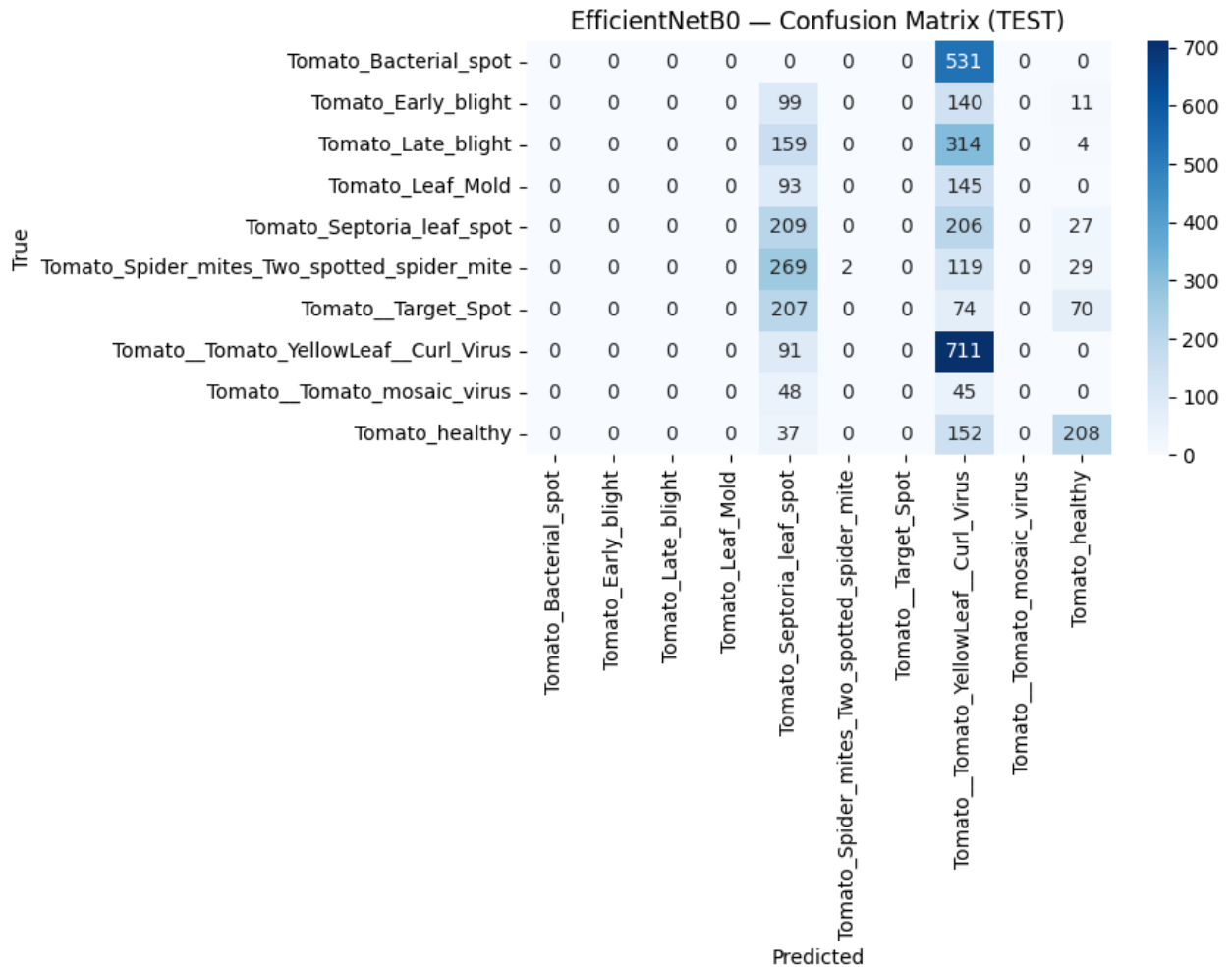




ResTEST Acc: 58.80%, Source: Imagenet

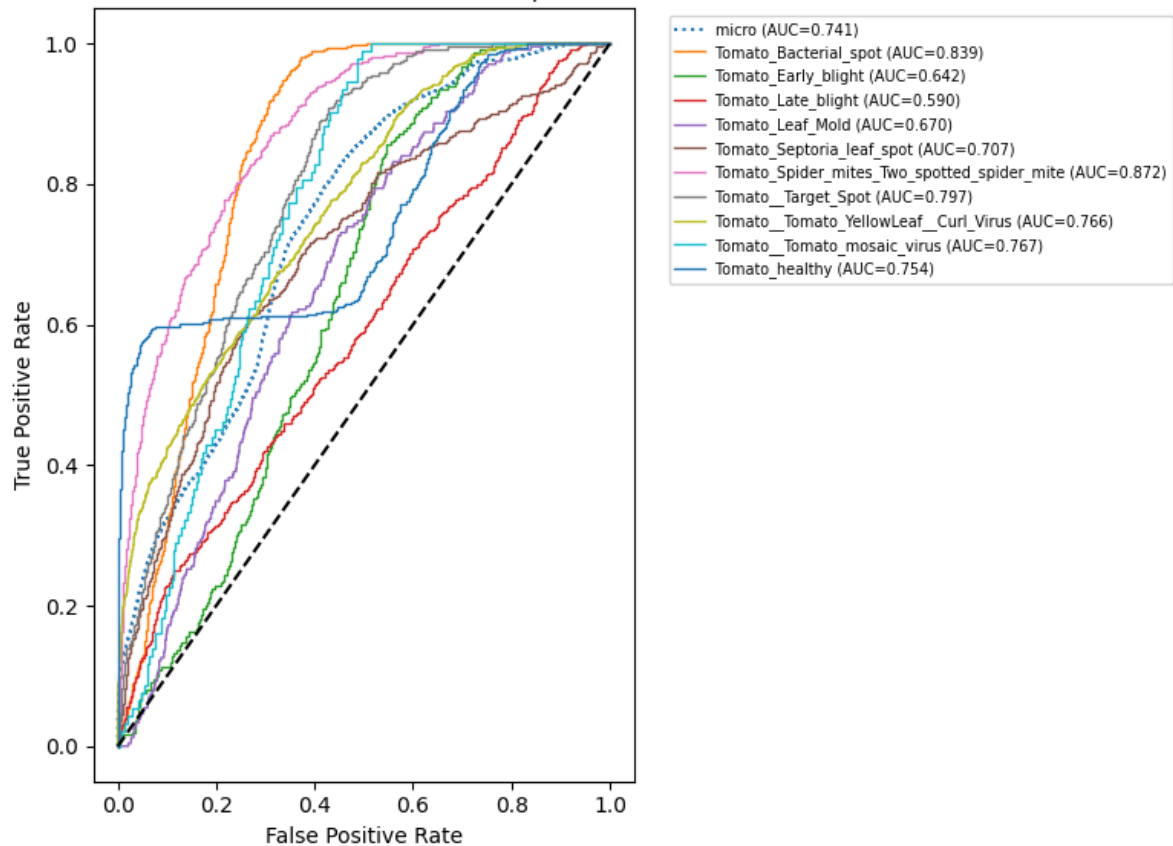
ResNet50 — ROC (TEST, first 10 classes) | macro AUC=0.909





EfficientNetB0 Acc:28.25%,Source:Imagenet

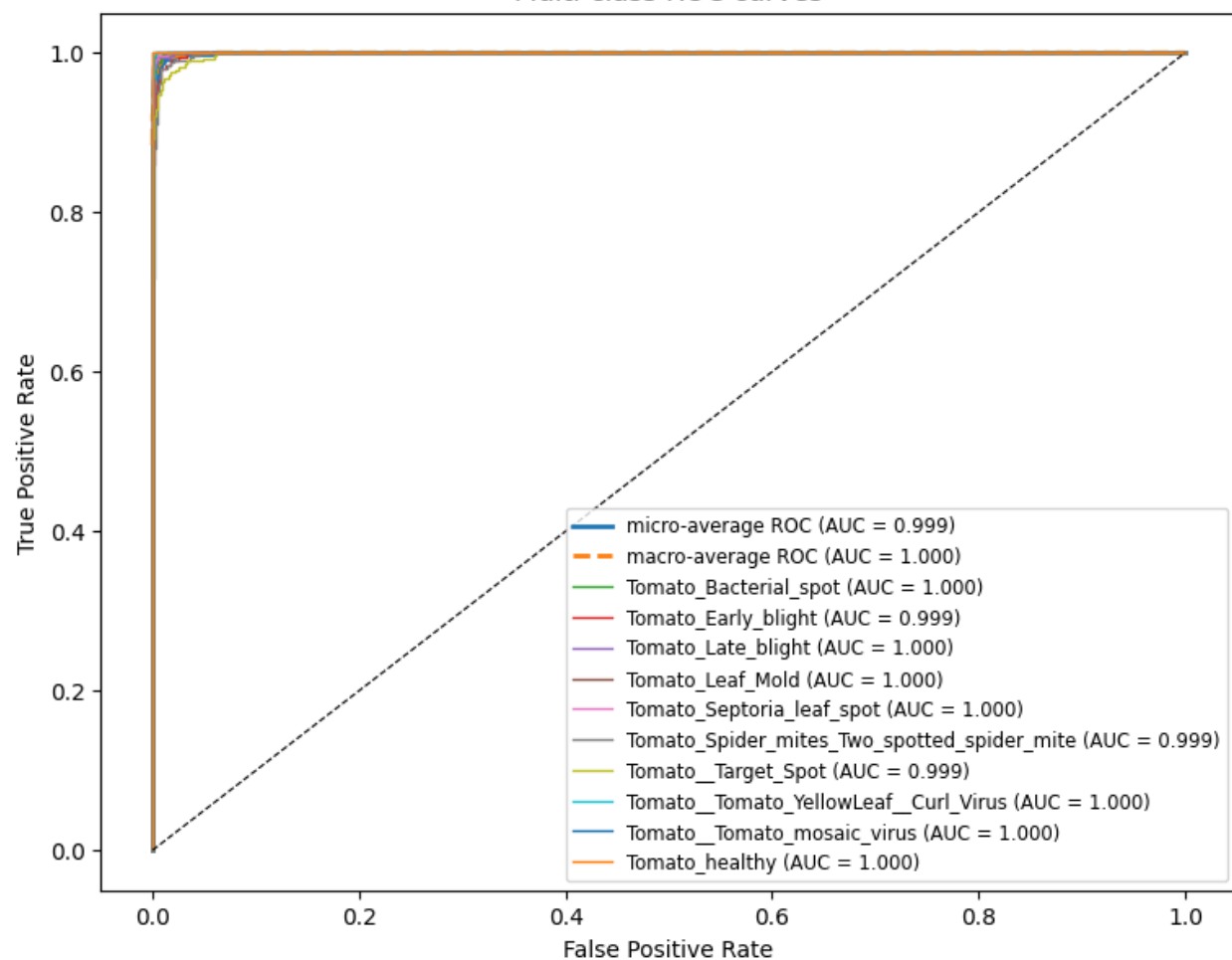
EfficientNetB0 — ROC (TEST, first 10 classes) | macro AUC=0.740

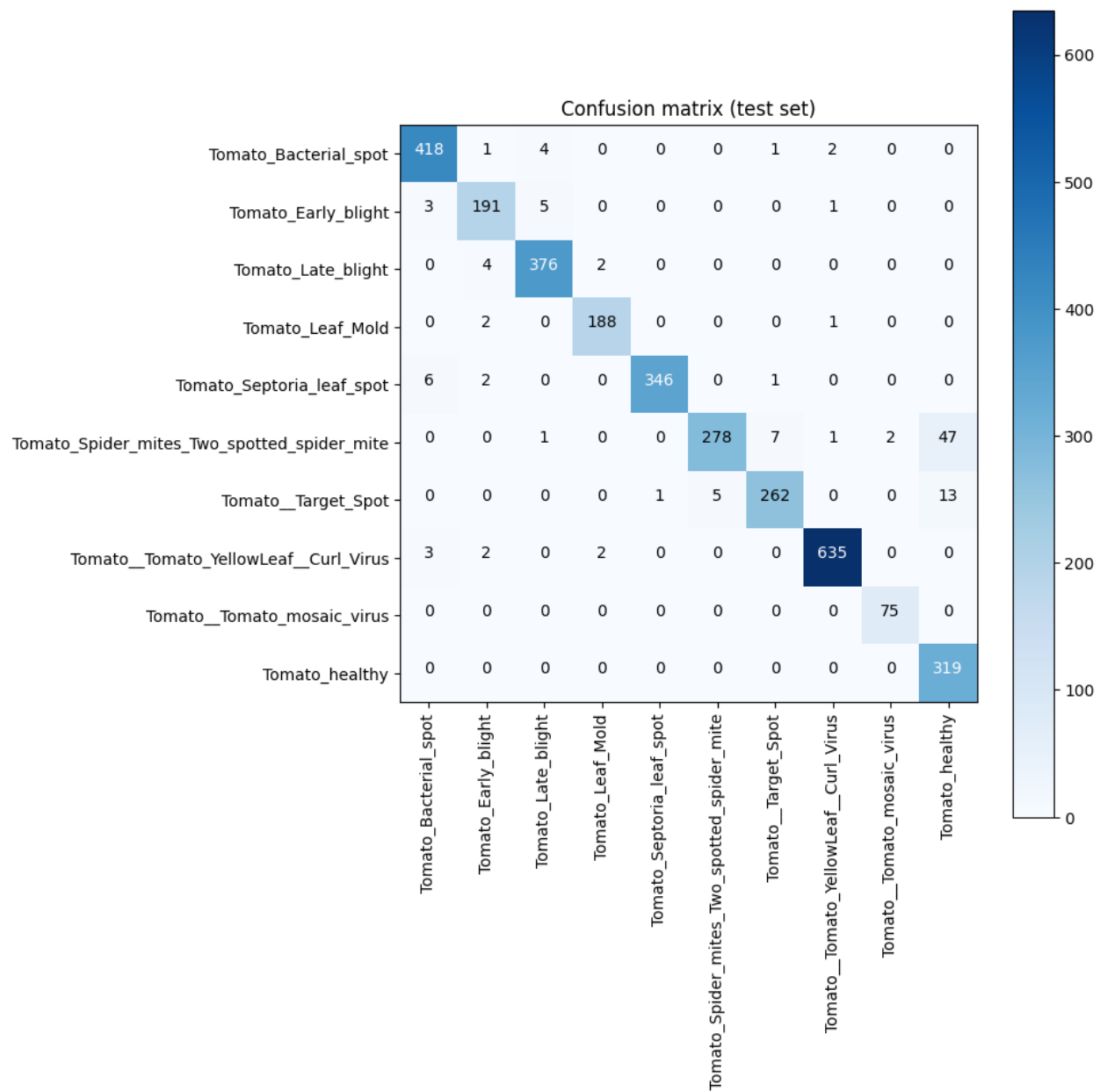


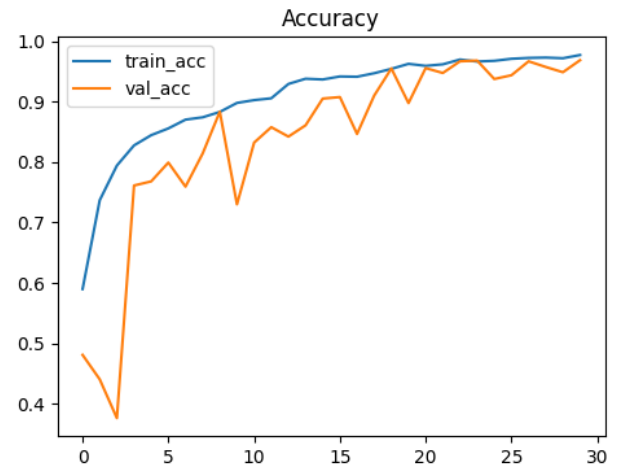
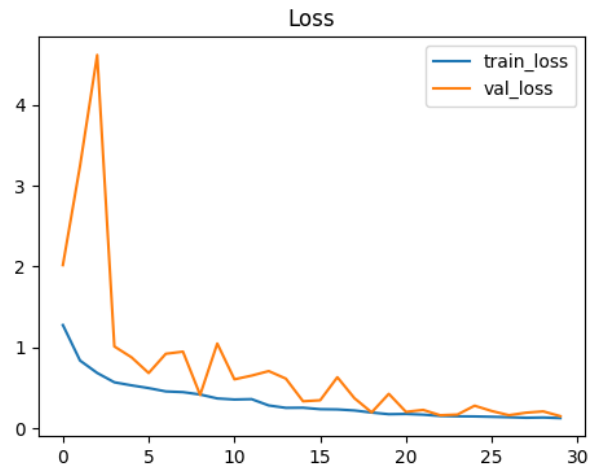
Task 3: We made a custom CNN . I am giving the layers ,

1. Input Layer (256*256*3)
2. Convolutional layer (Feature extraction). It detects edges, lines, color transitions in early stage and in deeper layer it will extract spots and other important things.
3. After each convolution, the **ReLU (Rectified Linear Unit)** activation function is applied.
4. Batch normalization for stable the learning process and speed.
5. Pooling layer for cutting unimportant things and keep only essentials.
6. Used dropout to prevent overfitting
7. Fully connected layer(Dense) for decision making.
8. Output layer for classifications.

Multi-class ROC curves

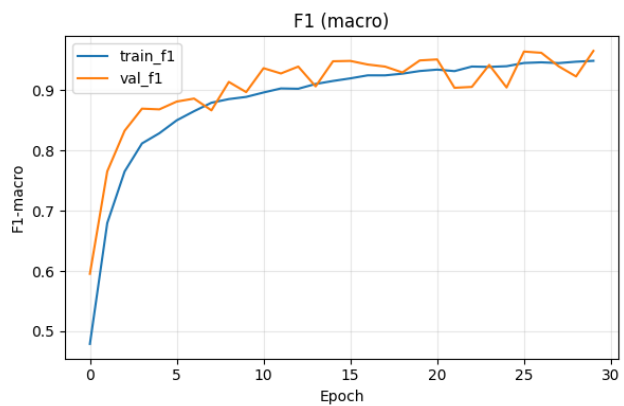
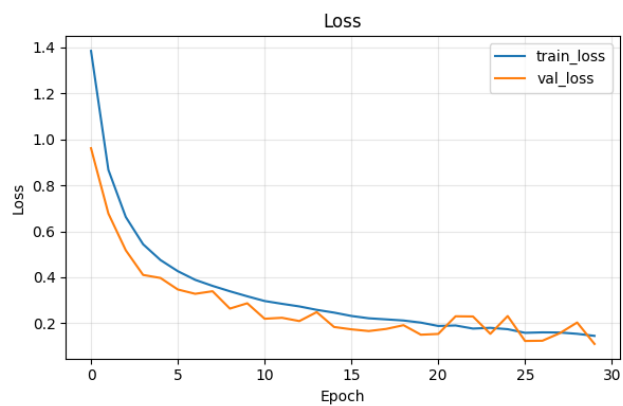
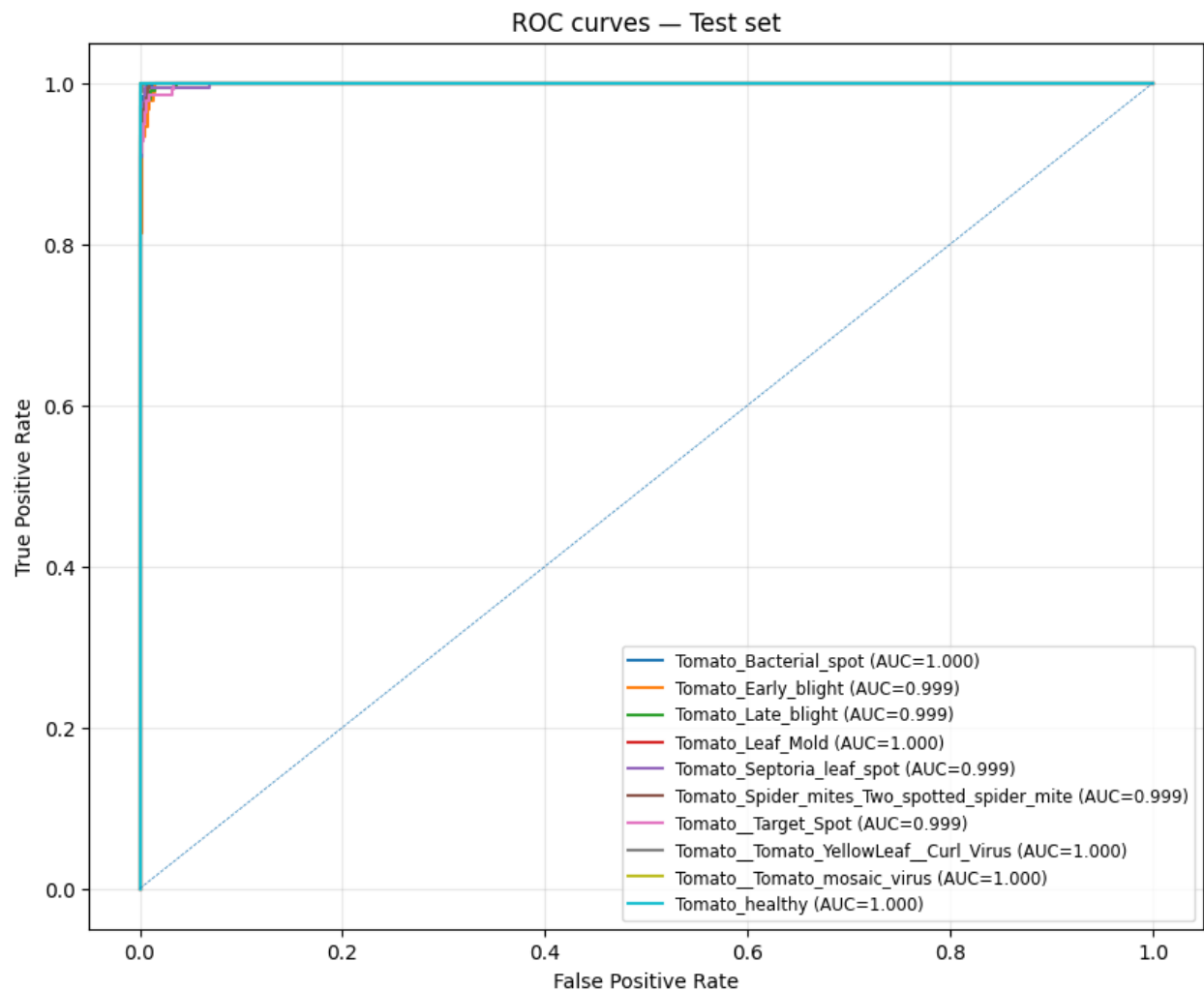






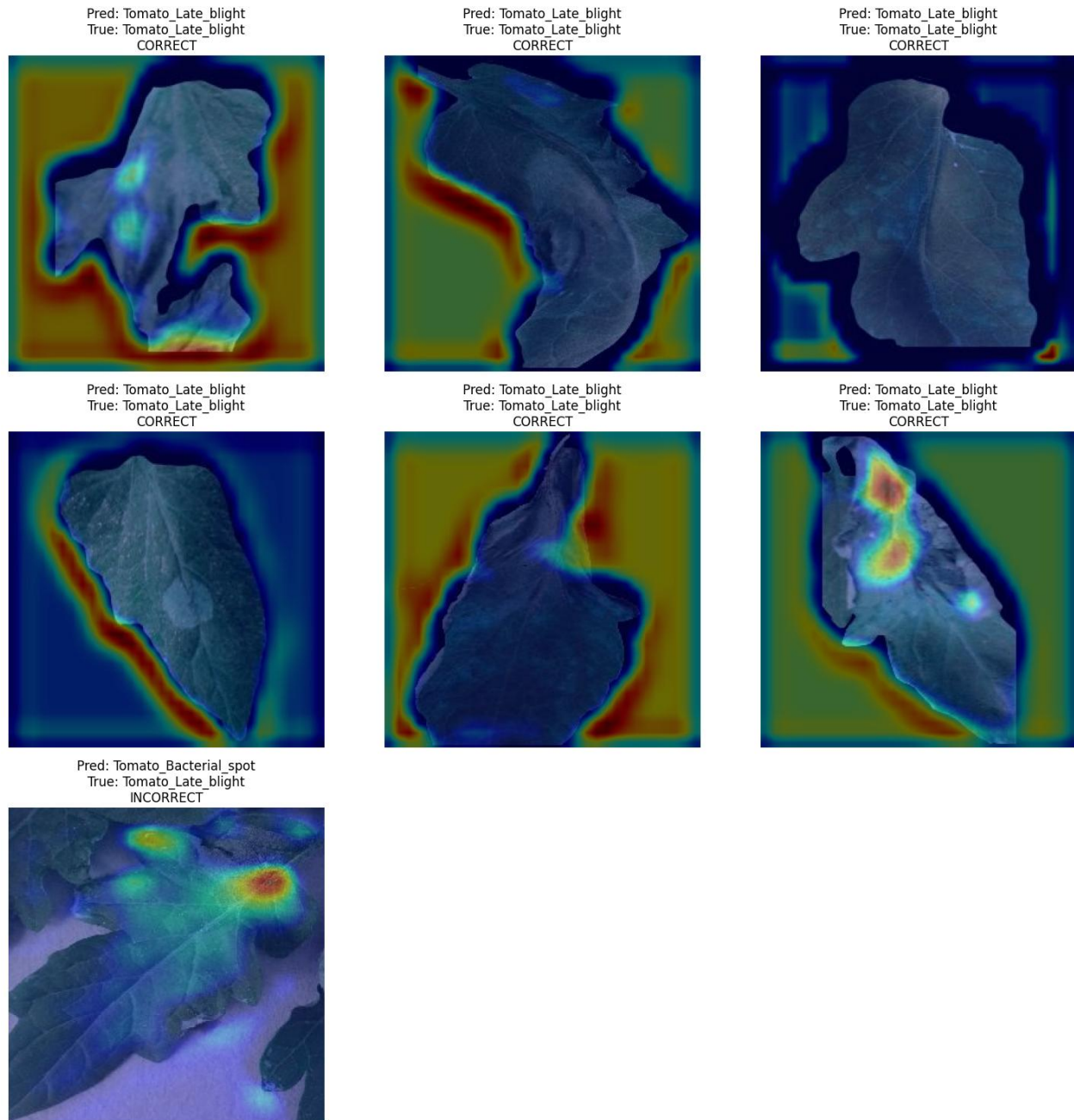
Accuracy : 96.28%

Task 4: Enhance the CNN model by integrating an attention module to refine feature extraction. We used CGNet for enhancement . Then Compare the metrics value with task2 and task3 .



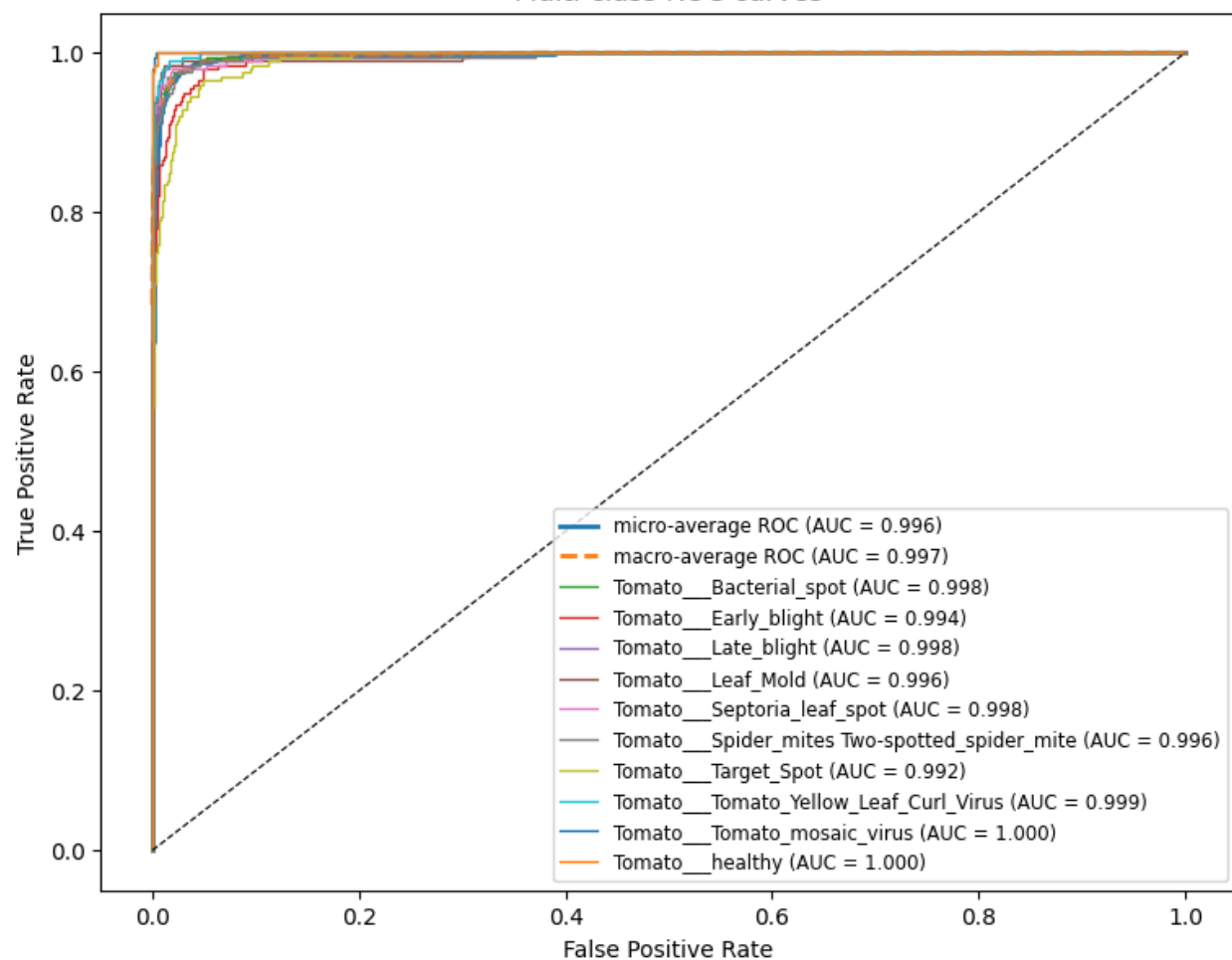
Enhanced Accuracy: 97.25%

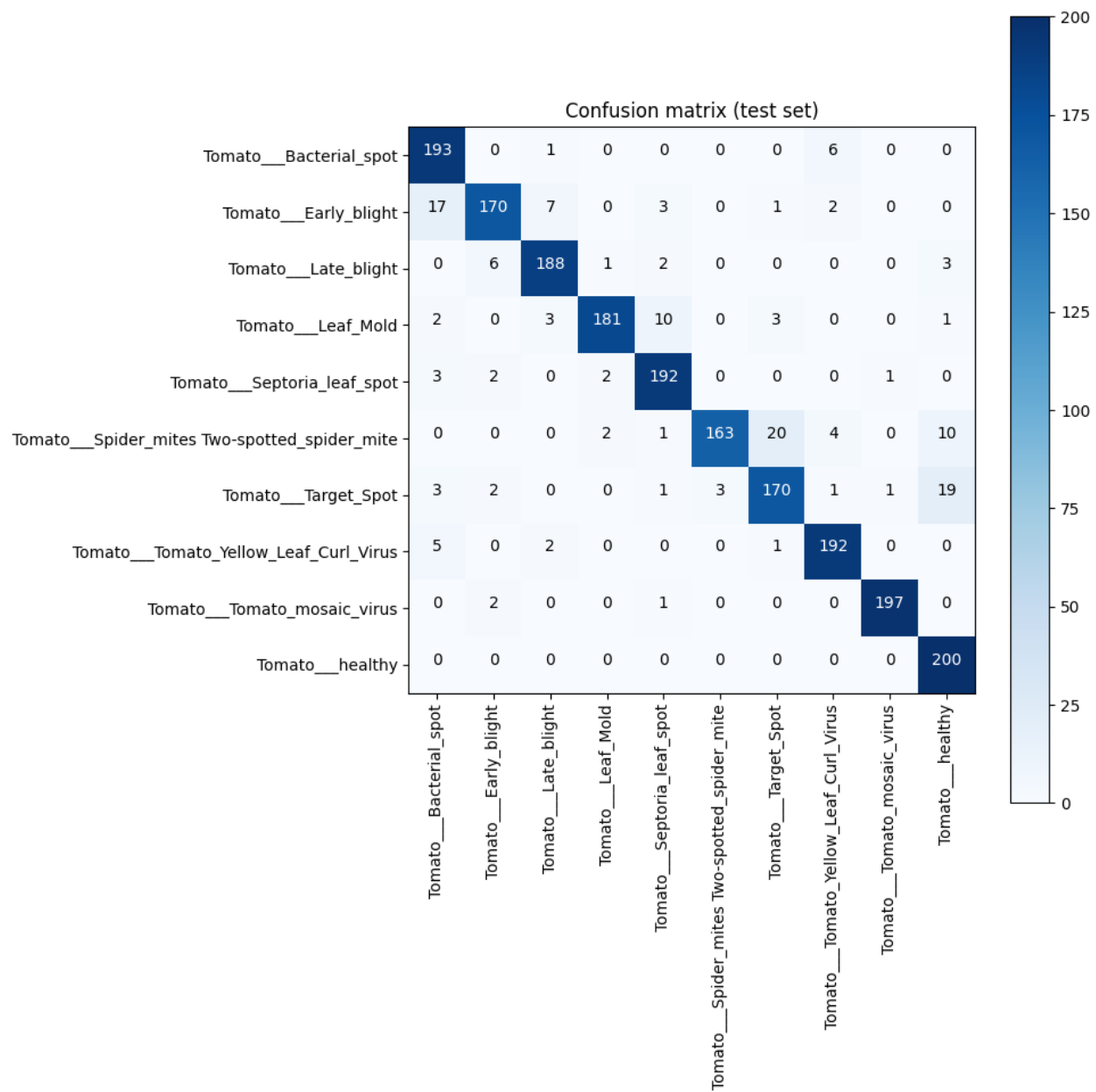
Task 5: Visualizing predictions, highlighting image regions for correct and incorrect samples using Grad-CAM. We also apply our custom CNN in different dataset of Tomato diseases.

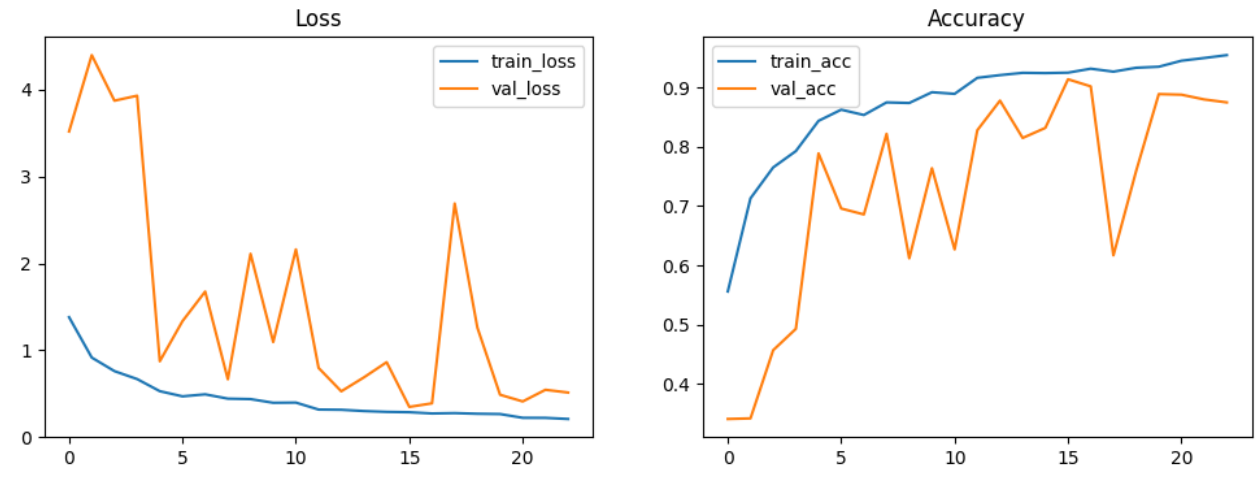


Now, for the another tomato diseases dataset

Multi-class ROC curves







2.4 Evalute Metrics :

Task 2 :

Model	Accuracy	Test AUC(macro ovr)	Test AUC (macro)	Source	Train Time(s)	Test Time(s)
VGG16	92.67%	0.995938	0.996939	Imagenet	5010.18	22.83
ResNet50	58.80%	0.909069	0.922079	Imagenet	3809.25	15.78
EfficientNetB0	28.25%	0.740417	0.741424	Imagenet	3023.96	14.90

Model A	Model B	t-stat	p-stat
VGG16	ResNet50	43.32	00000
VGG16	EfficientNetB0	82.11	00000
ResNet50	EfficientNetB0	34.01	6.458173e-223

Task 3: I am giving the structure of Custom CNN

Layer (type)	Output Shape	Param #
input_layer (InputLayer)	(None, 256, 256, 3)	0
conv2d (Conv2D)	(None, 256, 256, 32)	896
batch_normalization (BatchNormalization)	(None, 256, 256, 32)	128
activation (Activation)	(None, 256, 256, 32)	0
conv2d_1 (Conv2D)	(None, 256, 256, 32)	9,248
batch_normalization_1 (BatchNormalization)	(None, 256, 256, 32)	128
activation_1 (Activation)	(None, 256, 256, 32)	0
max_pooling2d (MaxPooling2D)	(None, 128, 128, 32)	0
dropout (Dropout)	(None, 128, 128, 32)	0
conv2d_2 (Conv2D)	(None, 128, 128, 64)	18,496
batch_normalization_2 (BatchNormalization)	(None, 128, 128, 64)	256
activation_2 (Activation)	(None, 128, 128, 64)	0
conv2d_3 (Conv2D)	(None, 128, 128, 64)	36,928
batch_normalization_3 (BatchNormalization)	(None, 128, 128, 64)	256
activation_3 (Activation)	(None, 128, 128, 64)	0
max_pooling2d_1 (MaxPooling2D)	(None, 64, 64, 64)	0
dropout_1 (Dropout)	(None, 64, 64, 64)	0
conv2d_4 (Conv2D)	(None, 64, 64, 128)	73,856
batch_normalization_4 (BatchNormalization)	(None, 64, 64, 128)	512
activation_4 (Activation)	(None, 64, 64, 128)	0
conv2d_5 (Conv2D)	(None, 64, 64, 128)	147,584

conv2d_5 (Conv2D)	(None, 64, 64, 128)	147,584
batch_normalization_5 (BatchNormalization)	(None, 64, 64, 128)	512
activation_5 (Activation)	(None, 64, 64, 128)	0
max_pooling2d_2 (MaxPooling2D)	(None, 32, 32, 128)	0
dropout_2 (Dropout)	(None, 32, 32, 128)	0
conv2d_6 (Conv2D)	(None, 32, 32, 256)	295,168
batch_normalization_6 (BatchNormalization)	(None, 32, 32, 256)	1,024
activation_6 (Activation)	(None, 32, 32, 256)	0
conv2d_7 (Conv2D)	(None, 32, 32, 256)	590,880
batch_normalization_7 (BatchNormalization)	(None, 32, 32, 256)	1,024
activation_7 (Activation)	(None, 32, 32, 256)	0
max_pooling2d_3 (MaxPooling2D)	(None, 16, 16, 256)	0
dropout_3 (Dropout)	(None, 16, 16, 256)	0
global_average_pooling2d (GlobalAveragePooling2D)	(None, 256)	0
dense (Dense)	(None, 512)	131,584
batch_normalization_8 (BatchNormalization)	(None, 512)	2,048
activation_8 (Activation)	(None, 512)	0
dropout_4 (Dropout)	(None, 512)	0
dense_1 (Dense)	(None, 256)	131,328
batch_normalization_9 (BatchNormalization)	(None, 256)	1,024
activation_9 (Activation)	(None, 256)	0
dropout_5 (Dropout)	(None, 256)	0
dense_2 (Dense)	(None, 10)	2,570

Accuracy	Macro precision	Macro recall	Macro F1	Train Time(s)	Test Time(s)
96.29%	0.962504	0.962832	0.961255	5763.16	9.41

Task 4:

Metric	Value
Accuracy	0.9725
Precision (Macro)	0.9666
Recall (Macro)	0.9728
F1-score (Macro)	0.9695
AUC (One-vs-Rest)	0.9996

Comparison with Task2 and and Task 3

Task	Model	Accuracy	F1 (Macro)	Training Time (s)	Inference/Test Time (s)
Task-2	VGG16	0.92675	0.91	5010.18	22.83
Task-2	ResNet50	0.58800	0.42	3809.25	15.78
Task-2	EfficientNetB0	0.28250	0.13	3023.96	14.89
Task-3	Custom CNN	0.96289	0.96126	5763.16	9.41
Task-4	Custom CNN + SE	0.97250	0.96950	1309.79	3.09

Task 5:

Dataset	Accuracy (%)	Macro Precision	Macro Recall	Macro F1-score	Training Time (s)	Testing Time (s)
Dataset 1 (PlantVillage)	95.60	0.95	0.95	0.95	4402.21	10.58
Dataset 2 (TomatoLeaf)	92.30	0.93	0.92	0.92	2790.60	9.89

3. Conclusion

This research demonstrates that custom CNN architectures, especially when augmented with GCNet, significantly improve the accuracy and efficiency of tomato leaf disease detection. The GCNet module enables better contextual feature understanding, leading to faster inference and robust classification. Future work may include deploying the model in real-time agricultural settings and expanding datasets for diverse environmental conditions.

References

- [1] Smith et al., “PlantVillage Dataset for Plant Disease Classification Using Simple CNN,” *IEEE Trans. on Agri. Informatics*, 2019.
- [2] Jones & Zhang, “Transfer Learning for PlantVillage Disease Classification Using EfficientNetB0,” *Springer Advances in Computer Vision*, 2020.
- [3] Patel et al., “Explainable AI for Field Crop Disease Detection Using Custom CNN and GradCAM,” *Elsevier Computers and Electronics in Agriculture*, 2022.
- [4] Li et al., “Vision Transformer for Agricultural Disease Recognition,” *IEEE Transactions on Image Processing*, 2021.
- [5] Wang et al., “Real-Time Rice Disease Detection Using YOLOv5 and EfficientNet,” *ACM Transactions on Applied AI*, 2022.
- [6] Kumar et al., “CropDeep: Multi-Crop Disease Recognition Using DenseNet-201,” *Elsevier Information Processing in Agriculture*, 2023.
- [7] Garcia & Lee, “TomatoLeafNet: MobileNetV3-Based Tomato Leaf Disease Classification,” *Springer AI in Agriculture*, 2024.
- [8] Consortium, “Federated Learning for Privacy-Preserving Crop Disease Recognition,” *IEEE Internet of Things Journal*, 2025.
- [9] M. Mohanty et al., “Using Deep Learning for Image-Based Plant Disease Detection,” *Frontiers in Plant Science*, 2018.
- [10] X. Hu et al., “Squeeze-and-Excitation Networks,” *IEEE CVPR*, 2018.
- [11] Y. Cao et al., “GCNet: Non-local Networks Meet Squeeze-Excitation Networks and Beyond,” *IEEE ICCV*, 2019.
- [12] K. Simonyan and A. Zisserman, “Very Deep Convolutional Networks for Large-Scale Image Recognition,” *arXiv preprint arXiv:1409.1556*, 2014.